

Metro Ticket Booking System

Solution Requirements Specification (ServiceNow Platform)

TARGET PLATFORM	DOCUMENT VERSION	ARTIFACT TYPE
ServiceNow App Engine	v1.0 (Solution Spec)	Functional & Non-Functional Requirements

1. Executive Overview

This document defines the functional, non-functional, data schema, and technical stack requirements for implementing an enterprise **Metro Ticket Booking System** on ServiceNow. The architecture leverages ServiceNow Service Catalog, Custom Tables extending task, Flow Designer, Integration Hub, and Scripted REST APIs for gate turnstile validation.

2. Functional Requirements (FRs)

FR1: User Portal & Mobile Catalog Experience

- **FR1.1 — Journey Selection:** The Service Portal / Now Mobile interface must allow commuters to select Source Station, Destination Station, Journey Date, and Ticket Type (Single Journey, Return Journey, Day Pass).
- **FR1.2 — Dynamic Fare Calculation:** The system must calculate the total fare dynamically based on the selected route matrix stored in `sn_metro_station` before order submission.
- **FR1.3 — Commuter Profile:** Users logged into the portal must be able to view their active tickets, past journey history, and stored pass balances.

FR2: Order & Ticket Lifecycle Management

- **FR2.1 — Ticket Generation:** Upon payment confirmation, the system must create a record in `sn_metro_ticket` (extending task) with a unique Ticket ID, valid timeframe, and state set to Active.
- **FR2.2 — QR Code Payload Creation:** The system must generate a unique, cryptographically signed payload (or encrypted JSON token) containing `Ticket_ID`, `Source_Station`, `Destination_Station`, and `Expiration_Time`.
- **FR2.3 — Ticket Expiration Automation:** Scheduled jobs or Flow Designer timers must automatically transition ticket states from Active to Expired if not used within the validity window (e.g., 3 hours from purchase).

FR3: Payment Integration

- **FR3.1 — Payment Processing:** Integration Hub REST steps must communicate with external payment gateways (e.g., Stripe, Razorpay, UPI) to process payment requests.
- **FR3.2 — Transaction Logging:** All transaction references, authorization codes, and status updates must be logged into `sn_metro_payment` for financial auditing.
- **FR3.3 — Automated Refund Logic:** If a ticket fails to issue after successful payment deduction, Flow Designer must automatically trigger a refund API call.

FR4: Turnstile / Entry-Exit Gate Validation

- **FR4.1 — Scanner API Endpoint:** ServiceNow must expose an inbound Scripted REST API (`/api/x_metro/gate_validation`) to process HTTP POST requests sent by turnstile hardware scanners.
- **FR4.2 — Real-time Validation:** The API must decrypt/verify the scanned QR token, check if `Ticket_State == Active`, verify station validity, and return a binary ALLOW / DENY response.
- **FR4.3 — Ticket Closure:** Upon valid entry scanning, the ticket status must update to `In-Transit`; upon valid exit scanning, it must update to `Closed/Used`.

3. Non-Functional Requirements (NFRs)

NFR1: Performance & Response Time

- **Gate Latency:** The Scripted REST API endpoint for turnstile validation must process and respond to gate scan requests in **under 200 milliseconds** to prevent queue bottlenecks at metro gates.
- **Portal Load Time:** Service Portal catalog pages must render in under 2 seconds during peak morning/evening commuting hours.

NFR2: Security & Fraud Prevention

- **QR Anti-Replay / Anti-Duplication:** QR code payloads must include a time-based dynamic salt or nonce to prevent users from sharing screenshots of valid tickets.
- **Role-Based Access Control (RBAC):**
 - `sn_metro.user`: Access to book tickets and view personal history via Portal.
 - `sn_metro.gate_service`: Dedicated REST API service account restricted exclusively to gate validation endpoints.
 - `sn_metro.admin`: Full CRUD access to station matrices, tariff configuration, and reporting workspaces.

NFR3: Scalability & Availability

- **High Availability:** The solution must handle high-concurrency purchase spikes (e.g., morning rush hours) using asynchronous flow processing for non-critical tasks like sending email/SMS receipts.

4. Data Schema Requirements (ServiceNow Tables)

Table Name	Label	Parent Table	Key Fields
sn_metro_station	Metro Station	cmdb_ci / Standalone	Station Name, Code, Metro Line, Active Status, Zone
sn_metro_fare_matrix	Station Fare Matrix	Standalone	Source Station (Ref), Dest Station (Ref), Distance, Base Fare
sn_metro_ticket	Metro Ticket	task	Ticket Number, Commuter (sys_user), Route, Fare, QR Token, State (Draft, Active, In-Transit, Closed, Expired)
sn_metro_payment	Payment Transaction	Standalone	Ticket Ref, Gateway Transaction ID, Payment Method, Amount, Status

5. ServiceNow Technical Stack Mapping

- **User Interface:** Service Portal / Now Mobile App Engine (Catalog Item & Custom Widgets)
- **Workflow Automation:** Flow Designer (Triggered on payment status update)
- **External Integration:** Integration Hub (Payment Gateway Outbound REST)
- **Gate Hardware Integration:** Scripted REST API (Inbound Webhook)
- **Business Logic:** Script Includes & Async Business Rules